

Sistema di Gestione Segnalazioni Municipali - Comune di Baronissi (v5.1)

Ecosistema software modulare sviluppato in standard **C99** per la gestione, lo sfoltimento, l'indicizzazione e il riciclo geometrico delle segnalazioni di anomalia urbana.

L'architettura simula un **Database Relazionale a geometria rigida** basato su file piatti di testo, escludendo qualsiasi degradamento prestazionale indotto da scansioni orizzontali in $O(n)$ sugli archivi storici e implementando un totale isolamento dei dati tramite **Opaque Pointers (Information Hiding)**.

Caratteristiche Tecniche Principali

Persistenza Hardware ad Asimmetria Fisica

- Separazione netta tra:
 - cache operativa volatile **REPORT_BENCH_LINE byte**
 - archivio Master stazionario **REPORT_MASTER_LINE byte**
 - Presenza del flag di attivazione cella al byte offset `[REPORT_MASTER_LINE-2]`
-

Algoritmo di Riciclo Chirurgico dei Buchi in $O(1)$

- I record modificati o rimossi generano celle vuote marcate 'N'
- Gli indici fisici (disk row N) vengono salvati in uno:
 - **Stack LIFO** ausiliario
- Il server può:
 - sovrascrivere i buchi
 - riutilizzare gli slot
 - evitare scansioni orizzontali $O(n)$ per il check delle celle 'N'

Prestazione:

Tempo costante $O(1)$

Ricerca Logaritmica su File d'Indice AVL in $O(\log n)$

Gli indici prodotti dagli AVL auto-bilanciati vengono salvati come:

- Inorder Arrays Sinistra -> Radice -> Destra
- sequenze continue su disco

Ricerca eseguita tramite:

- salti logaritmici usando le funzioni *findUserId* e *findReportId*
- Questo garantisce accesso diretto ai file master di stato in $O(1)$

Geometrie:

File indice AVL	Dimensione Riga
User ID	21 byte
Report ID	22 byte

Dashboard Statistica Atomica in $O(1)$

Registro centrale statico system_total_report.txt:

- 11 byte per riga 10 byte per intero scritto come char + 1 byte '\n'
- 13 indicatori di sistema

Traccia:

```
#define REG_IDX_GLOBAL_ID      0 /* ID incrementale globale dei report */
#define REG_IDX_COUNTER_BENCH  1 /* Contatore corrente elementi in bench */
#define REG_IDX_NM_REPORT      2 /* Numero totale di report attivi nel sistema */
#define REG_IDX_STAT_OPEN      3 /* Totale pratiche in stato OPEN */
#define REG_IDX_STAT_PROGRESS  4 /* Totale pratiche in stato IN_PROGRESS */
#define REG_IDX_STAT_CLOSED    5 /* Totale pratiche in stato CLOSED */
#define REG_IDX_CAT_ROAD       6 /* Totale anomalie stradali */
#define REG_IDX_CAT_LIGHTING   7 /* Totale anomalie illuminazione */
#define REG_IDX_CAT_WASTE      8 /* Totale anomalie rifiuti */
```

```
#define REG_IDX_CAT_INFRASTRUCT 9 /* Totale anomalie impianti pubblici */
#define REG_IDX_CAT_OTHER      10 /* Totale anomalie generiche */
#define REG_IDX_AVL_REP_COUNT  11 /* Numero di nodi/righe in AVL Report ID */
#define REG_IDX_AVL_USR_COUNT  12 /* Numero di nodi/righe in AVL User ID */
```

Prestazione:

Rendering istantaneo O(1)

Organizzazione dei File e Geometrie Rigide

```
database/
  Master_Files/
    users.txt
      Anagrafica Utenti (10(USER_ID) + MAX_USERNAME + MAX_PASSWORD + 1(Role))
    users_idx.txt
      Indice Hash Utenti (6 byte fissi)
    open_reports.txt
      Master Segnalazioni Aperte (10(REPORT_ID)+ 10(USER_ID) + MAX_USERNAME
      + 1(Report_status) + 10(disk_row) + 1(flag_cella)+ 1("\n")byte)
    in_progress_reports.txt
      Master Segnalazioni in Lavorazione (10(REPORT_ID)+ 10(USER_ID) + MAX_USERNAME
      + 1(Report_status) + 10(disk_row) + 1(flag_cella)+ 1("\n")byte)
    closed_reports.txt
      Master Segnalazioni Chiuse (10(REPORT_ID)+ 10(USER_ID) + MAX_USERNAME
      + 1(Report_status) + 10(disk_row) + 1(flag_cella)+ 1("\n")byte)
    open_holes.txt
      Stack LIFO Buchi Open (11 byte) (10byte disk row + "\n")
    in_progress_holes.txt
      Stack LIFO Buchi Progress (11 byte) (10byte disk row + "\n")
    closed_holes.txt
```

Stack LIFO Buchi Closed (11 byte) (10byte disk row + '\n')

system_total_report.txt

Registro Statistico Centrale (13 righe da 11 byte)

Derived_Files/

reports_bench.txt

Cache Operativa Server (LIMIT_BENCH slot da REPORT_BENCH_LINE byte)

report_AVL_BY_REPORT_ID.txt

Indice Ordinato per Report ID (22 byte) (10(REPORT_ID) + 1(status) + 10(disk row))

report_AVL_BY_USER_ID.txt

Indice Ordinato per User ID (21 byte)(10(USER_ID) + 10(REPORT_ID) + 1(\n))

Architettura della Memoria a Tre Livelli

1. RAM Volatile di Sessione (Cittadino)

Gestita tramite:

- ReportList
 - lista concatenata semplice
 - inserimento in testa
- ReportStack
 - array statico
 - undo profondo
 - massimo 10 elementi report

Le modifiche RAM avvengono solo sui report:

OPEN

2. Cache Vettoriale Statica (BENCH)

Al logout:

- i record vengono riversati in:
 - `reports_bench.txt`

Geometria:

REPORT_BENCH_LINE byte per slot
 $(10(\text{REPORT_ID}) + 10(\text{USER_ID}) + \text{MAX_USERNAME} + 1(\text{Report_category}) + \text{MAX_DESC} + 10)$

Comportamento:

- se il report esiste:
 - la cella viene riscritta sul posto
- gli incrementi anagrafici vengono azzerati

Al raggiungimento di:

`LIMIT_BENCH` slot

si attiva automaticamente:

Flush Pesante tramite `*process_and_flush_bench()*`

3. Database Master e Rigenerazione Indici

Il flush:

- smista i record validi nei Master
- ricicla i buchi tramite Stack LIFO per l'inserimento intelligente
- elimina logicamente i record `DESTROYED` non inserendoli nei Master, dati sporchi momentanei in Bench

Successivamente:

- il server rilegge i Master
- rigenera gli indici AVL
- esegue visite:
 - simmetriche
 - `In-Order`

Aggiorna inoltre:

- contatori posizionali
 - metadati statistici
-

Compilazione ed Esecuzione

Il progetto adotta:

- architettura modulare
- standard rigoroso C99
- nessuna dipendenza esterna

Compilazione

```
gcc -Wall -Wextra -pedantic -std=c99 -include src/main.c src/models/user.c src/models/
```

Esecuzione Interattiva

```
./segnalazioni_municipali.exe
```

Validazione del Sistema

Per validare:

- integrità logica
- consistenza hardware
- correttezza asintotica
- strutture dati
- pipeline AVL
- riciclo geometrico

eseguire la suite di test integrata.
